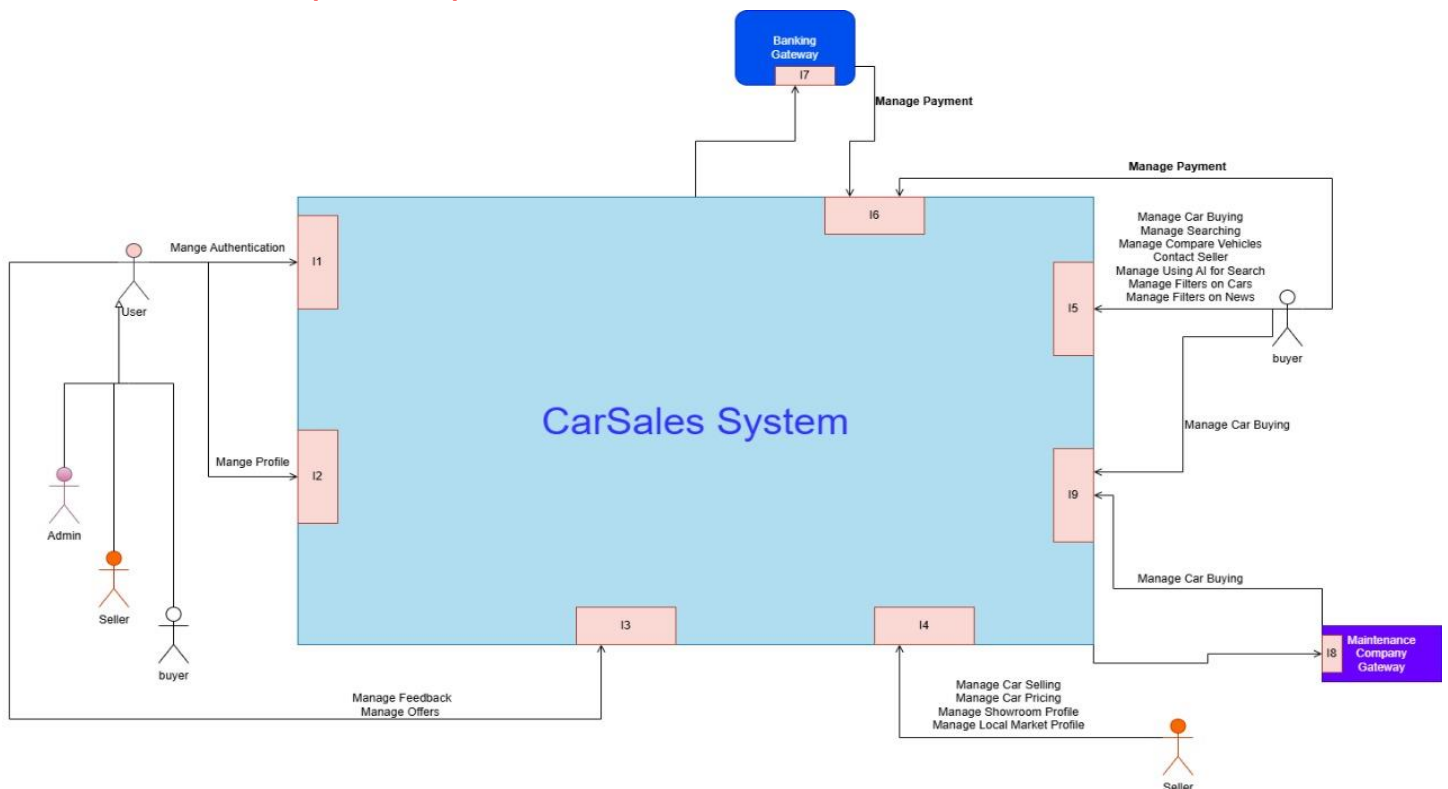


## Assignment 2 solution

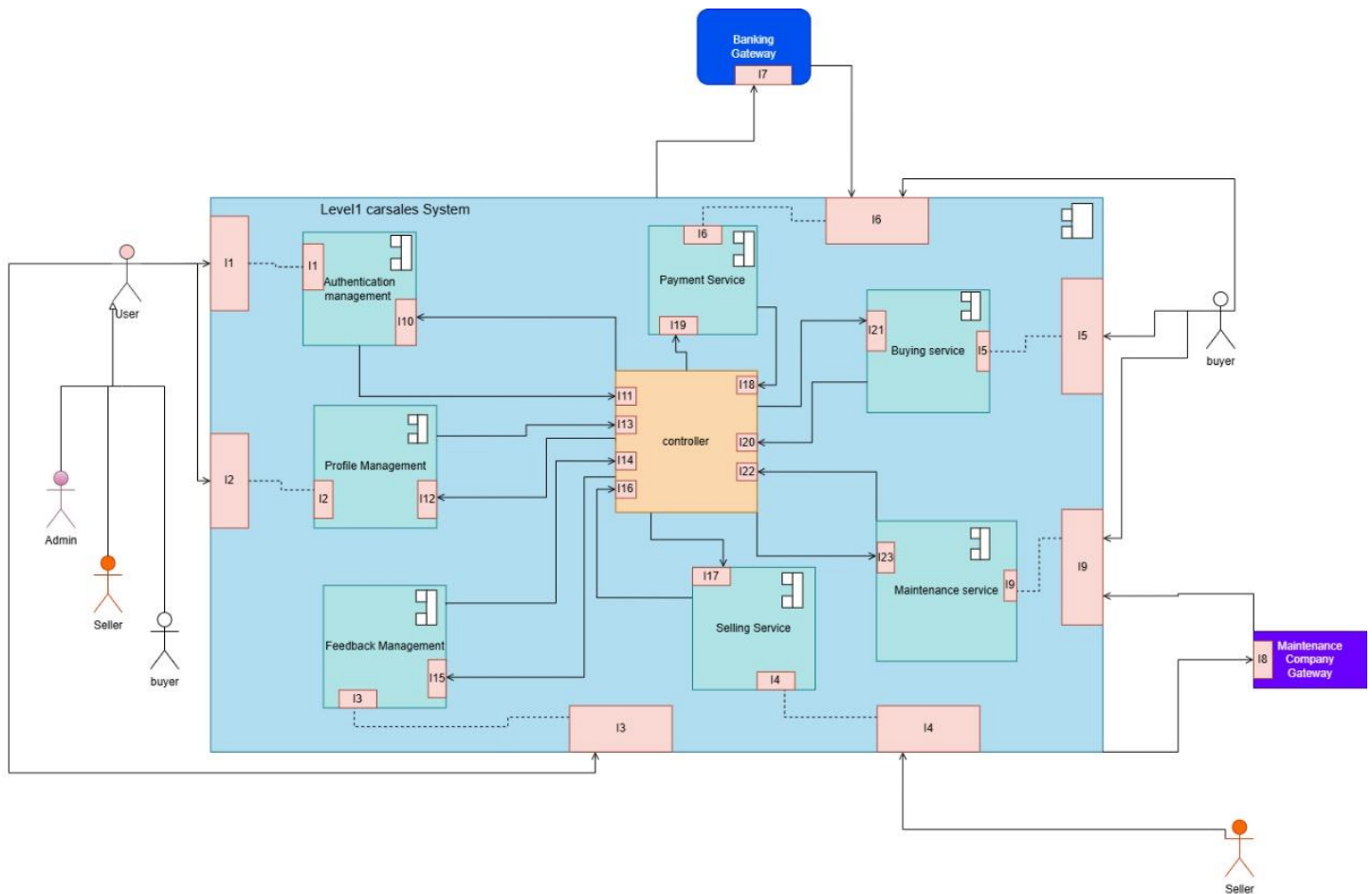
| ID       | Name                         | Email                                | Role               |
|----------|------------------------------|--------------------------------------|--------------------|
| 23102384 | Mennatullah Abdelaziz        | Mennatallah.abdelaziz.2024@aiu.ed.eg | Project Manager    |
| 23100209 | Hany Ziad Abdelaal Mohamed   | hany.mohamad.2024@aiu.edu.eg         | UI&UX              |
| 23101471 | Mohamed Mossad Ghonam        | mohamed.ghonam.2024@aiu.edu.eg       | Frontend Developer |
| 23101498 | Hoda Mahmoud Saad Abdelhalim | hoda.abdelhalim.2024@aiu.edu.eg      | Backend Developer  |
| 23101372 | Basmala Tarek Mahmoud Elkady | basmala.elkady.2024@aiu.edu.eg       | Backend Developer  |

### 1. Architecture Diagrams:

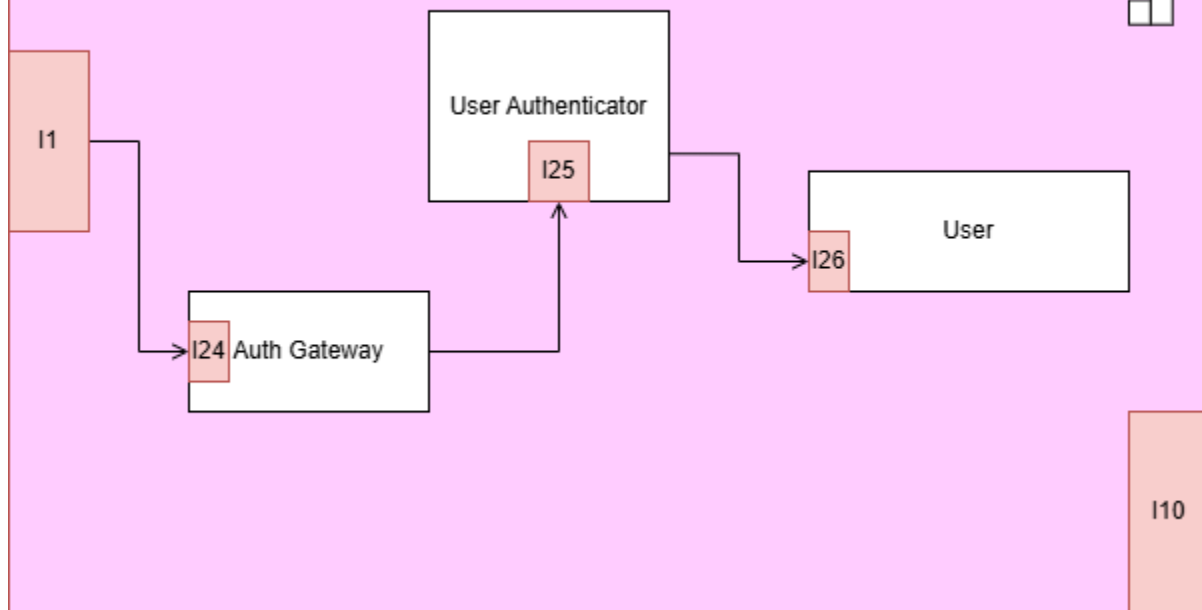
- Level 0 (context):



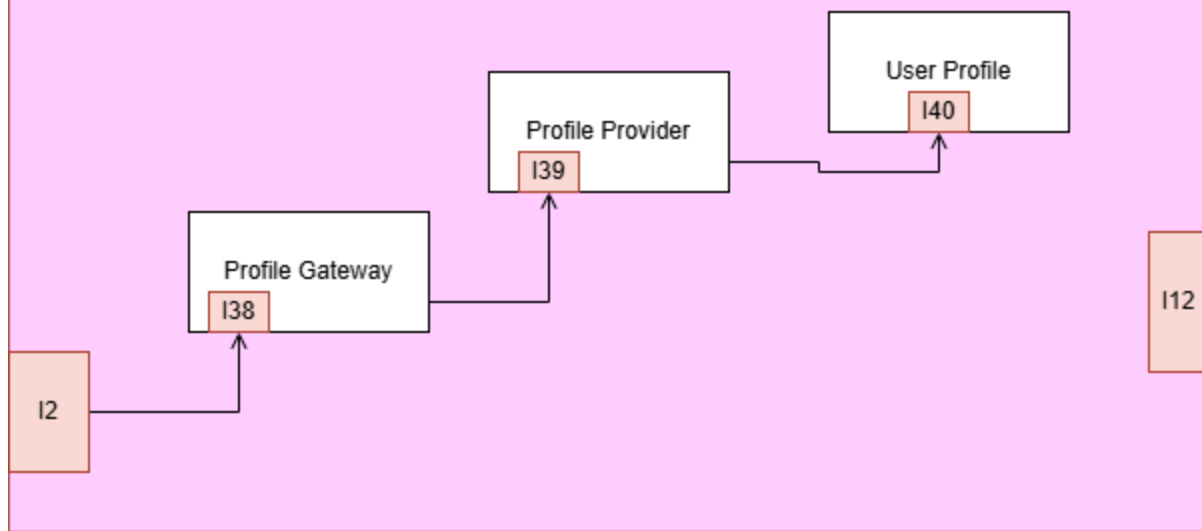
- Level 1:
- Level 2:



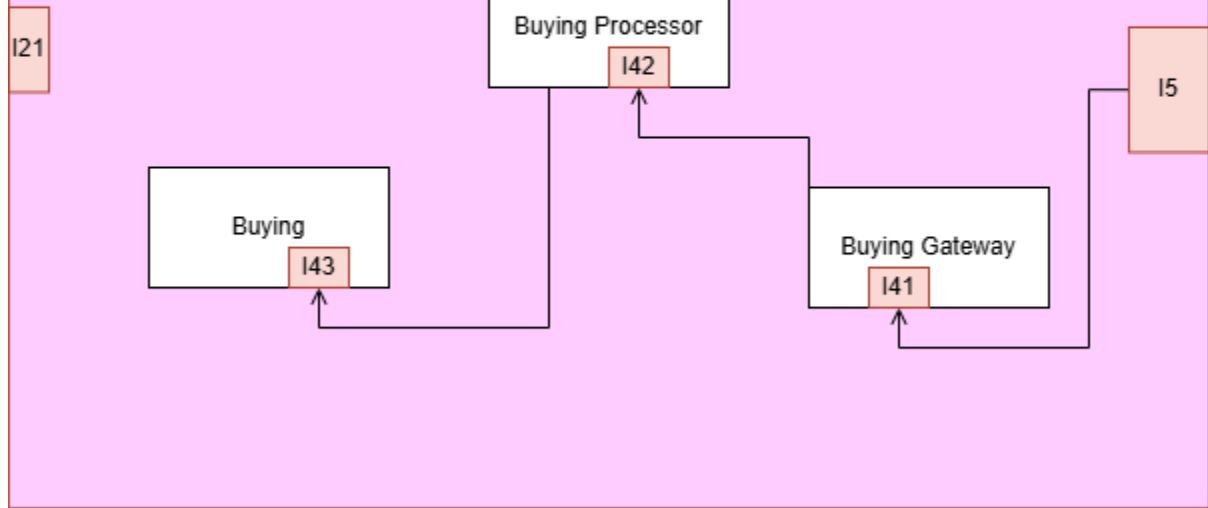
## Level 2 Authentication management



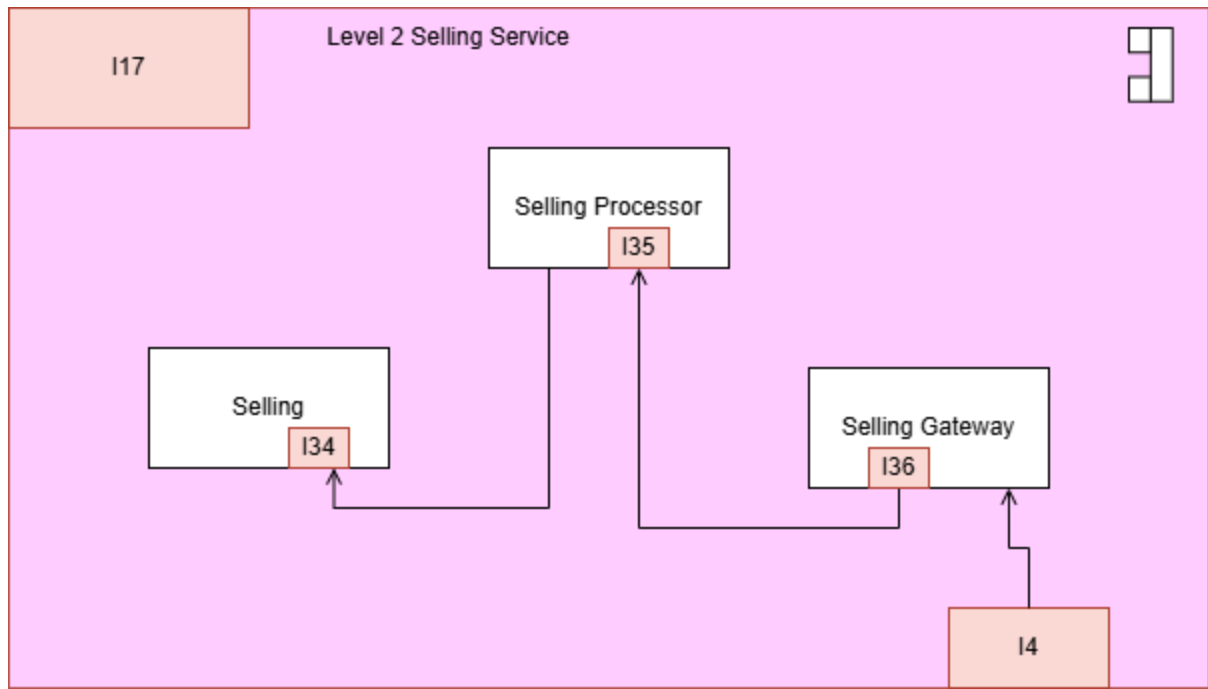
## Level 2 Profile Management



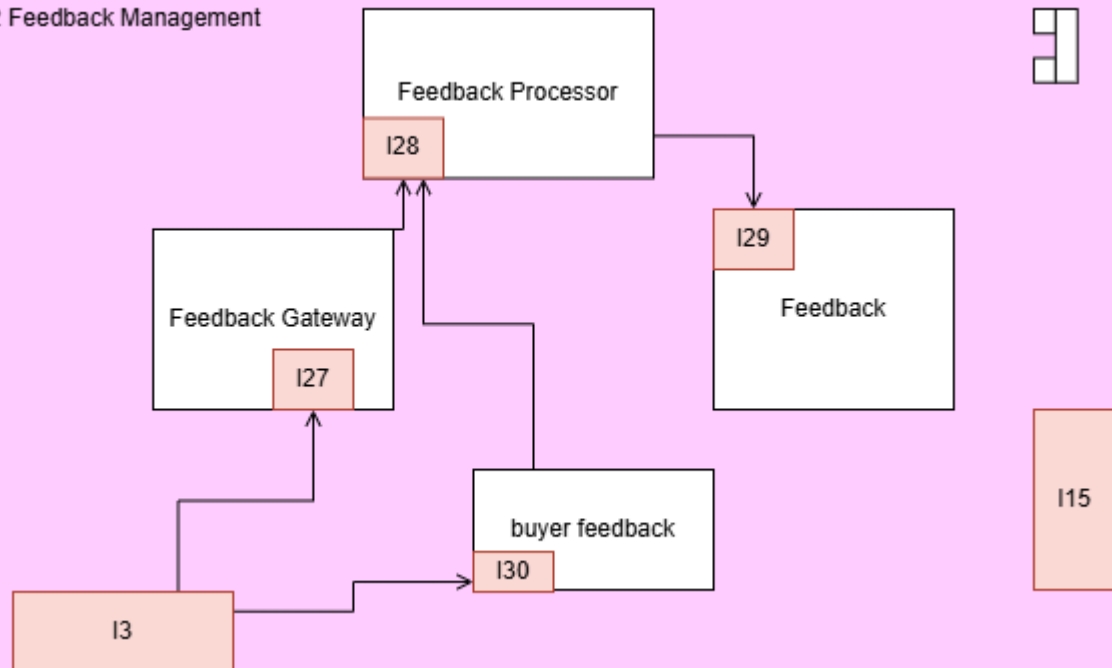
Level 2 Buying service



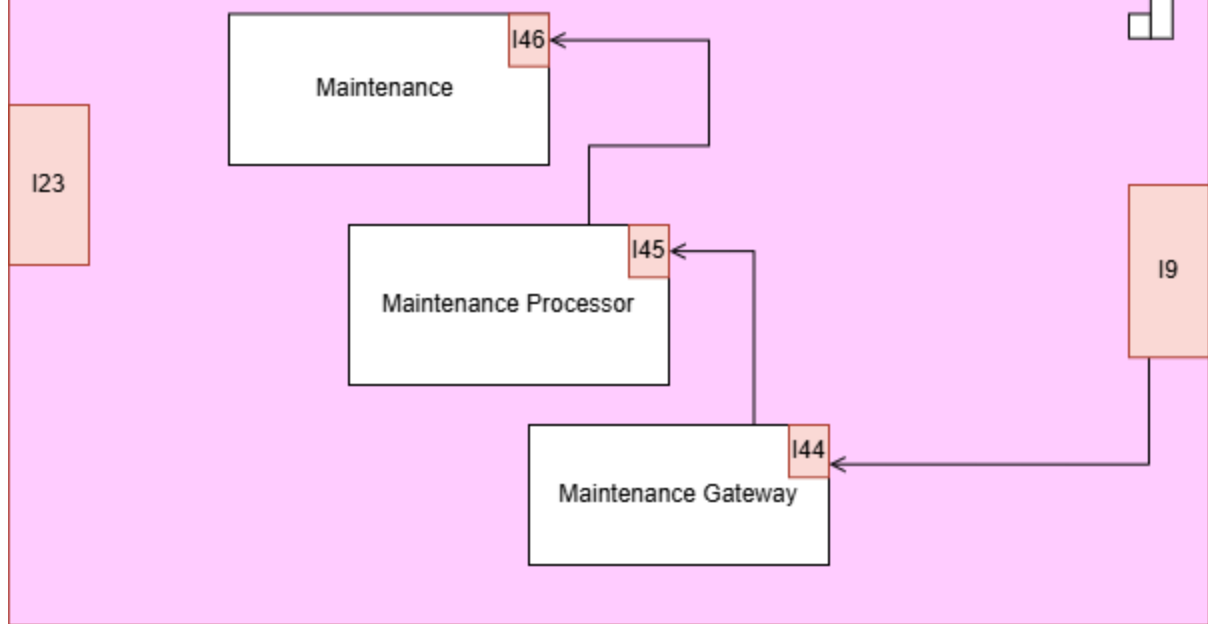
Level 2 Selling Service



## Level 2 Feedback Management

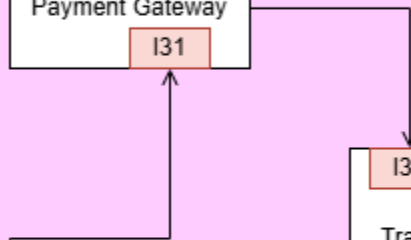
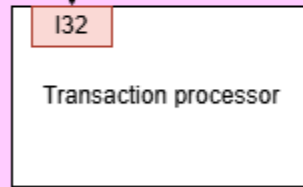
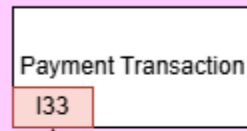
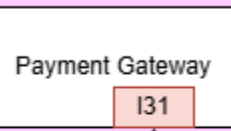


## Level 2 Maintenance services



Level 2 Payment Service

I6



## **2. interface definitions:**

### **Interface Descriptions for Car Buying and Selling System**

#### **I1: Authentication Management Interface — IAuthManagement**

**Name: Authentication Management Service Interface**

**Description: Manages user authentication and session handling for secure access.**

##### **Low-Level Functions:**

- **authenticateUser(UserCredentials credentials : AuthToken)**
- **validateSession(AuthToken token : boolean)**
- **logoutUser(AuthToken token : LogoutConfirmation)**
- **resetPassword(Email email, SecurityQuestion[] answers : ResetToken)**
- **changePassword(UserID id, OldPassword oldPass, NewPassword newPass : ChangeConfirmation)**
- **verifyEmail(UserID id, VerificationCode code : boolean)**
- **refreshToken(AuthToken oldToken : AuthToken)**
- **getUserRole(UserID id : UserRole)**

#### **I2: Profile Management Interface — IProfileManagement**

**Name: Profile Management Service Interface**

**Description: Handles creation, update, and management of user profiles with verification.**

##### **Low-Level Functions:**

- **createProfile(ProfileData data : ProfileID)**
- **getProfile(UserID id : ProfileData)**
- **updateProfile(UserID id, PartialProfileData newData : UpdateSummary)**
- **deleteProfile(UserID id : DeletionConfirmation)**
- **uploadDocuments(UserID id, List docs : UploadBatchResult)**

- **verifyIdentity(UserID id, IdentityProof proof : boolean)**
- **getUserStatus(UserID id : AccountStatus)**

### **I3: Feedback Management Interface — IFeedbackManagement**

**Name: Feedback Management Service Interface**

**Description: Collects, moderates, and retrieves ratings and written feedback for users and services.**

**Low-Level Functions:**

- **createFeedback(FeedbackData feedbackData : FeedbackID)**
- **listFeedbacks(FeedbackFilters filters : List)**
- **acceptFeedback(FeedbackID id : ModerationConfirmation)**
- **rejectFeedback(FeedbackID id, RejectionReason reason : RejectionConfirmation)**
- **updateFeedback(FeedbackID id, PartialFeedbackData newData : UpdateSummary)**

### **I4: Selling Service Interface — ISellingService**

**Name: Selling Service Interface**

**Description: Manages car selling operations for sellers, including listings and offer handling.**

**Low-Level Functions:**

- **createListing(CarData carDetails : ListingID)**
- **updateListing(ListingID id, PartialCarData updates : UpdateSummary)**
- **deleteListing(ListingID id : DeletionConfirmation)**
- **getListingsBySeller(SellerID sellerId, SearchFilter filter : List)**
- **receiveOffer(ListingID id, OfferData offer : OfferID)**
- **approveOffer(OfferID id : ApprovalConfirmation)**
- **rejectOffer(OfferID id, RejectionReason reason : RejectionConfirmation)**



- **markAsSold(ListingID id, TransactionID txId : boolean)**

## **I5: Buying Service Interface — IBuyingService**

**Name: Buying Service Interface**

**Description: Facilitates car discovery, offer submission, and buyer interactions like test drives.**

**Low-Level Functions:**

- **searchCars(SearchFilter filter : List)**
- **getCarDetails(ListingID id : CarData)**
- **sendOffer(BuyerID buyerId, ListingID listingId, OfferData offer : OfferID)**
- **cancelOffer(OfferID id : CancellationConfirmation)**
- **getBuyerOffers(BuyerID buyerId : List)**
- **requestTestDrive(ListingID id, BuyerID buyerId, ScheduleRequest schedule : TestDriveID)**

## **I6: Payment Service Interface — IPaymentService**

**Name: Payment Service Interface**

**Description: Processes payments, refunds, and fee calculations for transactions with audit trails.**

**Low-Level Functions:**

- **createPayment(PaymentData paymentInfo : PaymentID)**
- **verifyPayment(PaymentID id : boolean)**
- **processRefund(PaymentID id, RefundReason reason : RefundID)**
- **getPaymentStatus(PaymentID id : PaymentStatus)**
- **generateReceipt(PaymentID id : ReceiptDocument)**
- **calculateFees(PaymentData data, FeeType type : FeeBreakdown)**

## **I7: Loan Company Gateway Interface — IBankingGetway**

**Name: Loan Company Gateway Service Interface**

**Description: Integrates loan requests with external bank gateway.**

**Low-Level Functions:**

- **submitLoanRequest(UserID userId, LoanRequest req : RequestID)**
- **checkEligibility(RequestID id : EligibilityStatus)**
- **receiveOffer(RequestID id, LoanOffer offer : OfferID)**
- **confirmLoan(OfferID id : boolean)**

## **I8: Maintenance Company Gateway Interface — IMaintenanceCompanyGateway**

**Name: Maintenance Company Gateway Service Interface**

**Description: Integrates with maintenance companies for service requests and payments.**

**Low-Level Functions:**

- **submitMaintenanceRequest(UserID userId, MaintenanceRequest req : RequestID)**
- **checkAvailability(RequestID id : AvailabilityStatus)**
- **confirmService(RequestID id : ServiceID)**
- **processMaintenancePayment(ServiceID id : PaymentStatus)**

## **I9: Maintenance Service Interface — IMaintenanceService**

## **Name: Maintenance Service Interface**

**Description:** Manages car maintenance operations, including booking services, inspections, history tracking, warranty handling, and related add-ons like towing and part replacements.

### **Low-Level Functions:**

- **bookMaintenance(CarID carId, MaintenanceData data : BookingID)**
- **getMaintenanceHistory(CarID carId : MaintenanceHistoryReport)**
- **requestInspection(CarID carId : InspectionID)**
- **getInspectionReport(InspectionID id : InspectionReport)**
- **checkWarrantyStatus(CarID carId : WarrantyStatus)**
- **purchaseExtendedWarranty(CarID carId, WarrantyPlan plan : WarrantyID)**
- **getServiceQuotes(CarID carId, ServiceType type : List<QuoteSummary>)**
- **requestTowing(CarID carId, Location location : TowingID)**
- **scheduleOilChange(CarID carId, ScheduleRequest schedule : BookingID)**
- **calculateRepairEstimate(RepairData repairInfo : EstimateBreakdown)**

## **I10: Authentication Controller Response Interface — IAuthControllerResp**

### **Name: Authentication Controller Response Service Interface**

**Description:** Delivers authentication operation outcomes to the controller layer.

### **Low-Level Functions:**

- **loginResult(AuthToken result : void)**
- **sessionStatus(UserSession status : void)**
- **logoutResult(LogoutConfirmation result : void)**
- **passwordResetStatus(ResetToken status : void)**
- **passwordChangeStatus(ChangeConfirmation status : void)**
- **emailVerificationStatus(boolean isVerified : void)**

- **refreshTokenResult(AuthToken result : void)**
- **getUserRoleResult(UserRole result : void)**

## **I11: Authentication Controller Request Interface — IAuthController**

**Name: Authentication Controller Request Service Interface**

**Description:** Sends authentication management requests from the controller to the service layer for processing.

**Low-Level Functions:**

- **AuthenticateUser(UserCredentials credentials : AuthToken)**
- **ValidateSession(AuthToken token : boolean)**
- **LogoutUser(AuthToken token : LogoutConfirmation)**
- **ResetPassword(Email email, SecurityQuestion[] answers : ResetToken)**
- **ChangePassword(UserID id, OldPassword oldPass, NewPassword newPass : ChangeConfirmation)**
- **VerifyEmail(UserID id, VerificationCode code : boolean)**
- **RefreshToken(AuthToken oldToken : AuthToken)**
- **GetUserRole(UserID id : UserRole)**

## **I12: Profile Controller Response Interface — IProfileControllerResp**

**Name: Profile Controller Response Service Interface**

**Description:** Delivers profile management operation outcomes to the controller layer.

### **Low-Level Functions:**

- **createProfileResult(ProfileID result : void)**
- **getProfileResult(ProfileData result : void)**
- **updateProfileResult(UpdateSummary result : void)**
- **deleteProfileResult(DeletionConfirmation result : void)**
- **uploadDocumentsResult(UploadBatchResult result : void)**
- **verifyIdentityResult(boolean result : void)**
- **getUserStatusResult(AccountStatus result : void)**

### **I13: Profile Controller Request Interface — IProfileController**

**Name: Profile Controller Request Service Interface**

**Description:** Sends profile management requests from the controller to the service layer for processing.

### **Low-Level Functions:**

- **CreateProfile(ProfileData data : ProfileID)**
- **GetProfile(UserID id : ProfileData)**
- **UpdateProfile(UserID id, PartialProfileData newData : UpdateSummary)**
- **DeleteProfile(UserID id : DeletionConfirmation)**
- **UploadDocuments(UserID id, List docs : UploadBatchResult)**
- **VerifyIdentity(UserID id, IdentityProof proof : boolean)**
- **GetUserStatus(UserID id : AccountStatus)**

#### **I14: Feedback Controller Request Interface — IFeedbackController**

**Name: Feedback Controller Request Service Interface**

**Description: Sends feedback management requests from the controller to the service layer for processing.**

**Low-Level Functions:**

- **CreateFeedback(FeedbackData feedbackData : FeedbackID)**
- **ListFeedbacks(FeedbackFilters filters : List)**
- **AcceptFeedback(FeedbackID id : ModerationConfirmation)**
- **RejectFeedback(FeedbackID id, RejectionReason reason : RejectionConfirmation)**
- **UpdateFeedback(FeedbackID id, PartialFeedbackData newData : UpdateSummary)**

#### **I15: Feedback Controller Response Interface — IFeedbackControllerResp**

**Name: Feedback Controller Response Service Interface**

**Description: Delivers feedback management operation outcomes to the controller layer.**

**Low-Level Functions:**

- **createFeedbackResult(FeedbackID result : void)**
- **listFeedbacksResult(List result : void)**
- **acceptFeedbackResult(ModerationConfirmation result : void)**
- **rejectFeedbackResult(RejectionConfirmation result : void)**
- **updateFeedbackResult(UpdateSummary result : void)**

## **I16: Selling Controller Request Interface — ISellingController**

**Name: Selling Controller Request Service Interface**

**Description: Sends selling service requests from the controller to the service layer for processing.**

**Low-Level Functions:**

- **CreateListing(CarData carDetails : ListingID)**
- **UpdateListing(ListingID id, PartialCarData updates : UpdateSummary)**
- **DeleteListing(ListingID id : DeletionConfirmation)**
- **GetListingsBySeller(SellerID sellerId, SearchFilter filter : List)**
- **ReceiveOffer(ListingID id, OfferData offer : OfferID)**
- **ApproveOffer(OfferID id : ApprovalConfirmation)**
- **RejectOffer(OfferID id, RejectionReason reason : RejectionConfirmation)**

## **I17: Selling Controller Response Interface — ISellingControllerResp**

**Name: Selling Controller Response Service Interface**

**Description: Delivers selling service operation outcomes to the controller layer.**

**Low-Level Functions:**

- **createListingResult(ListingID result : void)**
- **updateListingResult(UpdateSummary result : void)**
- **deleteListingResult(DeletionConfirmation result : void)**
- **getListingsBySellerResult(List result : void)**

- **receiveOfferResult(OfferID result : void)**
- **approveOfferResult(ApprovalConfirmation result : void)**
- **rejectOfferResult(RejectionConfirmation result : void)**

## **I18: Payment Controller Request Interface — IPaymentController**

**Name: Payment Controller Request Service Interface**

**Description: Sends payment service requests from the controller to the service layer for processing.**

**Low-Level Functions:**

- **CreatePayment(PaymentData paymentInfo : PaymentID)**
- **VerifyPayment(PaymentID id : boolean)**
- **ProcessRefund(PaymentID id, RefundReason reason : RefundID)**
- **GetPaymentStatus(PaymentID id : PaymentStatus)**
- **GenerateReceipt(PaymentID id : ReceiptDocument)**
- **CalculateFees(PaymentData data, FeeType type : FeeBreakdown)**

## **I19: Payment Controller Response Interface — IPaymentControllerResp**

**Name: Payment Controller Response Service Interface**

**Description: Delivers payment service operation outcomes to the controller layer.**

**Low-Level Functions:**

- **createPaymentResult(PaymentID result : void)**



- **verifyPaymentResult(boolean result : void)**
- **processRefundResult(RefundID result : void)**
- **getPaymentStatusResult(PaymentStatus result : void)**
- **generateReceiptResult(ReceiptDocument result : void)**
- **calculateFeesResult(FeeBreakdown result : void)**

## **I20: Buying Controller Request Interface — IBuyingController**

**Name: Buying Controller Request Service Interface**

**Description: Sends buying service requests from the controller to the service layer for processing.**

**Low-Level Functions:**

- **SearchCars(SearchFilter filter : List)**
- **GetCarDetails(ListingID id : CarData)**
- **SendOffer(BuyerID buyerId, ListingID listingId, OfferData offer : OfferID)**
- **CancelOffer(OfferID id : CancellationConfirmation)**
- **GetBuyerOffers(BuyerID buyerId : List)**
- **RequestTestDrive(ListingID id, BuyerID buyerId, ScheduleRequest schedule : TestDriveID)**

## **I21: Buying Controller Response Interface — IBuyingServiceResp**

**Name: Buying Controller Response Service Interface**

**Description: Delivers buying service operation outcomes to the controller layer.**

### **Low-Level Functions:**

- **searchCarsResult(List result : void)**
- **getCarDetailsResult(CarData result : void)**
- **sendOfferResult(OfferID result : void)**
- **cancelOfferResult(CancellationConfirmation result : void)**
- **getBuyerOffersResult(List result : void)**
- **requestTestDriveResult(TestDriveID result : void)**

## **I22: Maintenance Controller Interface — IMaintenanceController**

**Name: Maintenance Controller Service Interface**

**Description: Sends maintenance service requests from the controller to the service layer for processing.**

### **Low-Level Functions:**

- **BookMaintenance(CarID carId, MaintenanceData data : BookingID)**
- **GetMaintenanceHistory(CarID carId : MaintenanceHistoryReport)**
- **RequestInspection(CarID carId : InspectionID)**
- **GetInspectionReport(InspectionID id : InspectionReport)**
- **CheckWarrantyStatus(CarID carId : WarrantyStatus)**
- **PurchaseExtendedWarranty(CarID carId, WarrantyPlan plan : WarrantyID)**
- **GetServiceQuotes(CarID carId, ServiceType type : List<QuoteSummary>)**
- **RequestTowing(CarID carId, Location location : TowingID)**
- **ScheduleOilChange(CarID carId, ScheduleRequest schedule : BookingID)**
- **CalculateRepairEstimate(RepairData repairInfo : EstimateBreakdown)**

## **I23: Maintenance Service Response Interface — IMaintenanceServiceResp**

**Name: Maintenance Service Response Service Interface**

**Description: Delivers maintenance service operation outcomes to the controller layer.**

### **Low-Level Functions:**

- **bookMaintenanceResult(BookingID result : void)**
- **getMaintenanceHistoryResult(MaintenanceHistoryReport result : void)**
- **requestInspectionResult(InspectionID result : void)**
- **getInspectionReportResult(InspectionReport result : void)**
- **checkWarrantyStatusResult(WarrantyStatus result : void)**
- **purchaseExtendedWarrantyResult(WarrantyID result : void)**
- **getServiceQuotesResult(List<QuoteSummary> result : void)**
- **requestTowingResult(TowingID result : void)**
- **scheduleOilChangeResult(BookingID result : void)**
- **calculateRepairEstimateResult(EstimateBreakdown result : void)**

## **I24: AuthGateway interface— IAuthGateway**

**Name: AuthGateway Service Interface**

**Description: Provides secure integration with external authentication providers for token exchange, session validation, and federated login support.**

### **Low-Level Functions:**

- **exchangeToken(ExternalToken token : InternalAuthToken)**
- **validateSession(ExternalSessionId sessionId : boolean)**
- **generateRedirect(ProviderType type : RedirectUrl)**
- **handleCallback(CallbackData callback : AuthResponse)**

- **getStatus(ProviderType type : ProviderHealth)**

## **I25: UserAuthenticator Interface — IUserAuthenticator**

**Name: UserAuthenticator Service Interface**

**Description:** Defines core operations for authenticating users against internal databases, hashing passwords, and generating session tokens. Enforces security policies like rate limiting and multi-factor authentication (MFA).

**Low-Level Functions:**

- **authenticateCredentials(UserCredentials creds : AuthToken)**
- **hashPassword(String plainPass : HashedPassword)**
- **initiateMFA(UserID userId : MFAToken)**
- **verifyMFACode(UserID userId, String code : boolean)**
- **generateSessionToken(UserID userId : SessionToken)**
- **invalidateAllSessions(UserID userId : InvalidationCount)**

## **I26: User Interface — IUser**

**Name: User Interface**

**Description:** Defines the structure for a core user interface , encapsulating profile data, roles, and authentication state. Used across modules for user-centric operations like profile updates and permission checks.

**Low-Level Functions:**

- **updateEmail(String newEmail : UpdateConfirmation)**
- **addSession(SessionToken token : void)**
- **removeSession(SessionToken token : void)**
- **isActive() : boolean**

## **I27: FeedbackGateway Interface — IFeedbackGateway**

## **Name: FeedbackGateway Service Interface**

**Description:** Manages the ingress and egress of feedback data to external moderation services and analytics platforms. Handles API integrations for spam detection, sentiment analysis, and aggregation for user ratings.

### **Low-Level Functions:**

- **submitFeedbackToExternal(FeedbackData data : SubmissionStatus)**
- **detectSpam(FeedbackData data : SpamScore)**
- **publishApprovedFeedback(FeedbackID id : PublicationResult)**
- **archiveFeedback(FeedbackID id : ArchiveConfirmation)**
- **syncSentimentData(FeedbackID id : SentimentUpdate)**

## **I28: FeedbackProcessor Interface — IFeedbackProcessor**

### **Name: FeedbackProcessor Service Interface**

**Description:** Processes incoming feedback for validation, moderation, and computation of metrics like average ratings. Applies business rules for anonymity, escalation to admins, and conflict resolution.

### **Low-Level Functions:**

- **moderateFeedback(FeedbackData data : ModerationDecision)**
- **computeUserRating(UserID id : AverageRating)**
- **anonymizeFeedback(FeedbackData data : AnonymizedData)**
- **resolveConflicts(List<FeedbackID> ids : ResolutionSummary)**
- **generateTrendReport(DateRange range : TrendAnalysis)**

## **I29: Feedback Interface — IFeedback**

### **Name: Feedback Service Interface**

**Description:** Represents a single feedback instance with metadata for persistence, including timestamps, attachments, and linkage to users/transactions. Supports state changes like pending to approved.

### **Low-Level Functions:**

- **setStatus(ModerationStatus status : void)**
- **getLinkedTransaction() : TransactionID**
- **attachMedia(File media : AttachmentID)**
- **removeAttachment(AttachmentID id : void)**
- **serializeForDisplay() : DisplayableFeedback**

### **I30: BuyerFeedback Interface — IBuyerFeedback**

**Name: BuyerFeedback Service Interface**

**Description:** Handles buyer-specific feedback submission and viewing, integrating with transaction history for contextual ratings and reviews on purchases.

**Low-Level Functions:**

- **submitBuyerReview(TransactionID txId, ReviewData data : ReviewID)**
- **viewBuyerFeedbacks(BuyerID buyerId : List<FeedbackSummary>)**
- **rateSellerPostPurchase(SellerID sellerId, RatingScore score : RatingConfirmation)**
- **retrievePurchaseReviews(ListingID listingId : List<BuyerReview>)**
- **flagBuyerFeedback(FeedbackID id, FlagReason reason : FlagStatus)**

### **I31: PaymentGateway Interface — IPaymentGateway**

**Name: PaymentGateway Service Interface**

**Description:** Integrates with external payment providers for secure transaction routing, tokenization, and real-time status updates. Supports multiple gateways like Stripe or PayPal for failover and compliance.

**Low-Level Functions:**

- **routePayment(PaymentData data : GatewayResponse)**
- **tokenizeCard(CardDetails card : TokenID)**
- **processCharge(PaymentID id, Amount amount : ChargeResult)**

- **handleRefund(PaymentID id : RefundStatus)**
- **verifyGatewayStatus(GatewayType type : HealthCheck)**
- **syncTransactionLog(PaymentID id : SyncResult)**

### **I32: TransactionProcessor Interface — ITransactionProcessor**

**Name: TransactionProcessor Service Interface**

**Description:** Processes payment transactions internally, including validation, fee calculation, and audit logging. Ensures idempotency and handles edge cases like partial payments.

**Low-Level Functions:**

- **validateTransaction(PaymentData data : ValidationResult)**
- **calculateFees(PaymentData data : FeeBreakdown)**
- **executePayment(PaymentID id : ExecutionStatus)**
- **handlePartialPayment(PaymentID id, PartialAmount amount : PartialResult)**

### **I33: PaymentTransaction Interface — IPaymentTransaction**

**Name: PaymentTransaction Interface**

**Description:** Represents a single payment transaction interface with state management, including persistence, status transitions, and linkage to user accounts.

**Low-Level Functions:**

- **setStatus(TransactionStatus status : void)**
- **getLinkedUser() : UserID**
- **addFeeRecord(FeeData fee : void)**
- **removeFeeRecord(FeeID id : void)**
- **serializeForReport() : TransactionReport**

- **checkCompleteness() : boolean**

### **I34: Selling Interface — ISelling**

**Name: Selling Service Interface**

**Description:** Represents a core selling interface for car listings, encapsulating details like price, condition, and state transitions from draft to active/sold. Supports persistence and queries for seller views.

**Low-Level Functions:**

- **setStatus(ListingStatus status : void)**
- **getSellerId() : SellerID**
- **addCarDetail(CarData detail : void)**
- **removeCarDetail(CarID id : void)**
- **serializeForListing() : ListingData**
- **isActive() : boolean**

### **I35: SellingProcessor Interface — ISellingProcessor**

**Name: SellingProcessor Service Interface**

**Description:** Processes selling workflows including listing validation, offer handling, and sale closure. Applies rules for pricing adjustments, availability updates, and automated notifications.

**Low-Level Functions:**

- **validateListing(CarData data : ValidationResult)**
- **processOffer(OfferData offer : ProcessingStatus)**
- **closeSale(ListingID id : ClosureResult)**
- **adjustPrice(ListingID id, PriceAdjustment adj : AdjustmentConfirmation)**
- **notifySeller(OfferID offerId : NotificationStatus)**



- **updateInventory(SellerID sellerId : InventoryUpdate)**

### **I36: SellingGateway Interface — ISellingGateway**

**Name: SellingGateway Service Interface**

**Description: Integrates with external marketplaces and inventory systems for listing syndication, real-time syncing, and cross-platform offer routing.**

**Low-Level Functions:**

- **publishToExternal(ListingData data : PublishStatus)**
- **syncFromExternal(ListingID id : SyncResult)**
- **routeOfferToExternal(OfferData offer : RouteStatus)**
- **pollExternalStatus(ListingID id : StatusUpdate)**
- **withdrawFromExternal(ListingID id : WithdrawConfirmation)**
- **validateExternalLink(ListingID id : LinkValidation)**

### **I38: ProfileGateway Interface — IProfileGateway**

**Name: ProfileGateway Service Interface**

**Description: Integrates with external storage and identity services for profile data synchronization, document uploads, and secure retrieval from cloud providers or federated systems.**

**Low-Level Functions:**

- **syncProfileData(ProfileData data : SyncStatus)**
- **retrieveFromExternal(UserID id : ProfileData)**
- **uploadSecureDocument(File doc : UploadID)**
- **deleteExternalDocument(UploadID id : DeletionResult)**
- **validateSyncIntegrity(ProfileID id : IntegrityCheck)**

- **pollExternalUpdates(UserID id : UpdateNotification)**

### **I39: ProfileProvider Interface — IProfileProvider**

**Name: ProfileProvider Service Interface**

**Description: Provisions and manages profile resources, including data validation, consent handling, and provisioning workflows for new or updated profiles across user roles.**

**Low-Level Functions:**

- **provisionNewProfile(ProfileRequest req : ProfileID)**
- **validateProfileUpdate(PartialProfileData data : ValidationResult)**
- **handleConsent(UserID id, ConsentType type : ConsentStatus)**
- **generateProfileToken(ProfileID id : AccessToken)**

### **I40: UserProfile Interface — IUserProfile**

**Name: UserProfile Interface**

**Description: Represents a user profile interface with core attributes, state management, and methods for serialization and privacy controls.**

**Low-Level Functions:**

- **updateAttributes(PartialProfileData data : void)**
- **getPrivacySettings() : PrivacyConfig**

- **addVerification(DocumentRef doc : void)**
- **removeVerification(DocumentRef doc : void)**
- **isVerified() : boolean**

#### **I41: BuyingGateway Interface — IBuyingGateway**

**Name: BuyingGateway Service Interface**

**Description:** Integrates with external search engines, inventory APIs, and notification services for car discovery, real-time availability checks, and cross-platform offer submissions.

**Low-Level Functions:**

- **searchExternalCars(SearchFilter filter : ExternalResults)**
- **syncCarDetails(CarID id : SyncStatus)**
- **submitOfferToExternal(OfferData offer : SubmitResult)**
- **pollAvailability(ListingID id : AvailabilityUpdate)**
- **notifyExternalTestDrive(TestDriveID id : NotificationStatus)**
- **validateExternalSearch(SearchFilter filter : ValidationResult)**

#### **I42: BuyingProcessor Interface — IBuyingProcessor**

**Name: BuyingProcessor Service Interface**

**Description:** Processes buyer workflows including search refinement, offer validation, and interaction scheduling. Applies rules for personalization, budget checks, and automated recommendations.

**Low-Level Functions:**

- **refineSearch(SearchFilter filter : RefinedResults)**
- **validateOffer(OfferData offer : ValidationScore)**

- **scheduleTestDrive(TestDriveID id : ScheduleConfirmation)**
- **recommendCars(BuyerID buyerId : RecommendationList)**
- **processCancellation(OfferID id : CancellationStatus)**
- **calculateBudgetFit(CarData car, Budget budget : FitScore)**

### **I43: Buying Interface — IBuying**

**Name: Buying Service Interface**

**Description:** Represents a buyer interface with preferences, search history, and state management for ongoing purchases. Supports persistence and queries for personalized views.

**Low-Level Functions:**

- **updatePreferences(Preferences prefs : void)**
- **getSearchHistory() : HistoryList**
- **addFavorite(CarID carId : void)**
- **removeFavorite(CarID carId : void)**
- **isEligible() : boolean**

### **I44: MaintenanceGateway Interface — IMaintenanceGateway**

**Name: MaintenanceGateway Service Interface**

**Description:** Integrates with external maintenance providers, workshops, and inventory systems for service booking, real-time availability syncing, and API-based reporting.

**Low-Level Functions:**

- **bookExternalService(MaintenanceData data : BookingStatus)**

- **syncWorkshopAvailability(ServiceType type : AvailabilityList)**
- **submitInspectionRequest(InspectionData data : RequestID)**
- **retrieveExternalReport(ReportID id : ReportData)**
- **notifyServiceCompletion(ServiceID id : NotificationResult)**
- **validateGatewayConnection(ProviderType type : ConnectionHealth)**

#### **I45: MaintenanceProcessor Interface — IMaintenanceProcessor**

**Name: MaintenanceProcessor Service Interface**

**Description:** Processes maintenance workflows including booking validation, scheduling optimization, and cost estimation. Applies rules for priority queuing, part compatibility, and follow-up reminders.

**Low-Level Functions:**

- **validateBooking(MaintenanceData data : ValidationResult)**
- **optimizeSchedule(BookingID id, TimeSlot slot : OptimizedSlot)**
- **estimateCost(MaintenanceData data : CostEstimate)**
- **processServiceCompletion(ServiceID id : CompletionStatus)**
- **sendReminder(BookingID id : ReminderSent)**
- **checkPartCompatibility(CarID carId, PartData part : CompatibilityResult)**

#### **I46: Maintenance Interface — IMaintenance**

**Name: Maintenance Service Interface**

**Description:** Represents a maintenance interface with attributes, state transitions, and persistence for bookings, inspections, and history tracking.

**Low-Level Functions:**

- **updateRepairStatus(RepairStatus status : void)**
- **getCarModelAndYear() : CarDetails**
- **logMaintenanceAction(ActionData action : void)**
- **clearObsoleteNotes(NoteID id : void)**
- **generateServiceHistoryLog() : ServiceLogEntry**
- **verifyMaintenanceCompletion() : boolean**

## Interaction Scenario

### Mange Auth

| Actors' Interactions                                                                  | System Responses                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. User submits registration form with profile data (e.g., email, name, details).     | 2. Auth Management receives data, forwards to Main Controller; Main Controller calls createProfile on Profile Management, which saves the profile and returns Profile ID.                                              |
|                                                                                       | 3. Auth Management receives Profile ID, generates a verification code, and sends it automatically to the user's email (e.g., "Click to verify your email").                                                            |
| 4. User checks email, opens the link or enters the verification code in the app.      | 5. Auth Management receives the code input, sends user ID + code to Email Verification service for validation.                                                                                                         |
|                                                                                       | 6. Email Verification processes the request and returns status (isVerified: true/false); Auth Management updates profile flag and notifies user (e.g., success: "Email verified!" or error: "Invalid code – resend?"). |
| 7. User (with verified email) enters and submits initial password in the secure form. | 8. Auth Management receives password, hashes it, and sends changePassword request (user ID + new password) to Main Controller.                                                                                         |
|                                                                                       | 9. Main Controller updates password in Profile Management and returns change status                                                                                                                                    |

|                                                                                   |                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                   | (confirmation); Auth Management shows user message (e.g., "Password set successfully!").                                                                                                                                                   |
| 10. User enters email and password to attempt login.                              | 11. Auth Management receives credentials, forwards authenticateUser to Main Controller for validation against database.                                                                                                                    |
|                                                                                   | 12. Main Controller authenticates successfully, generates Auth Token, and returns loginResult (token) to Auth Management.                                                                                                                  |
|                                                                                   | 13. Auth Management immediately calls validateSession on Main Controller with the new token.                                                                                                                                               |
|                                                                                   | 14. Main Controller checks and returns sessionStatus (e.g., active user session details) to Auth Management.                                                                                                                               |
| 15. User starts using the app (e.g., navigates to dashboard).                     | 16. (If session active but token nearing expiry) Auth Management auto-triggers refreshToken with old token to Main Controller.                                                                                                             |
|                                                                                   | 17. Main Controller validates old token, generates new one, and returns refreshTokenResult (new Auth Token) to Auth Management for seamless continuation.                                                                                  |
| 18. User accesses a feature requiring permissions (e.g., viewing sales listings). | 19. Auth Management calls getUserRole on Main Controller with user ID.                                                                                                                                                                     |
|                                                                                   | 20. Main Controller queries Profile Management, returns getUserRoleResult (e.g., "buyer" or "seller") to Auth Management; Auth Management uses it to authorize and deliver full loginResult (token + role) to user, loading the dashboard. |



|                                                            |                                                                                                                                 |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>21. User selects "Logout" from the menu or profile.</b> | <b>22. Auth Management receives request with current Auth Token, forwards logoutUser to Main Controller.</b>                    |
|                                                            | <b>23. Main Controller invalidates the token, ends the session, and returns logoutResult (confirmation) to Auth Management.</b> |
|                                                            | <b>24. Auth Management notifies user (e.g., "Logged out safely.") and redirects to the login screen.</b>                        |

## **Mange Car Selling**

| <b>Actors' Interactions</b>                                                                      | <b>System Responses</b>                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Seller submits authenticateUser(credentials) to Auth Mgmt for login.</b>                   | <b>2. Auth Mgmt validates credentials securely.</b>                                                                                                                                           |
|                                                                                                  | <b>3. Auth Mgmt forwards verifyUserStatus(sellerID) to Profile Mgmt.</b>                                                                                                                      |
|                                                                                                  | <b>4. Profile Mgmt checks and returns userStatus(isVerified = true) to Auth Mgmt, which generates and returns loginResult(AuthToken) confirming loginSuccess and unlocking the dashboard.</b> |
| <b>5. Seller (authenticated) submits createListing(vehicleData) directly to Selling Service.</b> | <b>6. Selling Service validates vehicleData via validateListing(vehicleData).</b>                                                                                                             |
|                                                                                                  | <b>7. Selling Service serializes data for listing via serializeForListing() and returns to internal process.</b>                                                                              |

|                                                                                                                                                                                |                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                | <b>8. Selling Service processes createListing(vehicleData) and returns createListingResult(listingID).</b>                                                                                                |
|                                                                                                                                                                                | <b>9. Selling Service publishes the listing externally via publishToExternal(listingData) and updates inventory with updateInventory(sellerID), then notifies seller of success with listing details.</b> |
|                                                                                                                                                                                | <b>10. (If validation fails) Selling Service returns listingDenied(reason); notifies Seller of denial with specific error details and resubmit options.</b>                                               |
| <b>11. Seller requests getListingsBySeller(sellerID, filter) directly to Selling Service to view their listings.</b>                                                           | <b>12. Selling Service retrieves and returns getListingsBySellerResult(listings).</b>                                                                                                                     |
|                                                                                                                                                                                | <b>13. Selling Service processes and displays seller listings via displaySellerListings(listings) in an interactive dashboard view.</b>                                                                   |
| <b>(Asynchronous External Trigger - After listing publication) Complementary Services receives external offer on published listing via receiveOffer(listingID, offerData).</b> | <b>14. Complementary Services calls receiveOffer(listingID, offerData) to Selling Service.</b>                                                                                                            |
|                                                                                                                                                                                | <b>15. Selling Service processes offerData via processOffer(offerData) and routes to routeOfferToExternal(offerData).</b>                                                                                 |
|                                                                                                                                                                                | <b>16. External service returns receiveOfferResult(offerID); Selling Service</b>                                                                                                                          |

|                                                                                                                                         |                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                         | notifies Seller of new offer via notifySeller(offerID) with alert and details.                                                                                                                      |
| 17. Seller reviews received offer and submits approveOffer(offerID) directly to Selling Service.                                        | 18. Selling Service processes approval: notifySeller(offerID), closeListing(listingID), markAsSold(listingID, txID), and returns approveOfferResult(approvalConfirmation).                          |
|                                                                                                                                         | 19. Selling Service sends offerApproved(approvalConfirmation) back to Seller, including deal summary and next steps.                                                                                |
| 20. (Post-approval) Seller or system triggers scheduleMaintenance(carID, date) directly to Complementary Services for the sold vehicle. | 21. Complementary Services executes processBookingRequest(carID, date).                                                                                                                             |
|                                                                                                                                         | 22. Complementary Services executes externalBooking(bookingData) and returns externalBookingSuccess(refID).                                                                                         |
|                                                                                                                                         | 23. Complementary Services saves appointment(refID, carID) via saveAppointment(refID, carID) and displays appointmentTicket(ticketID) to Seller for sharing, including validateExternalLink(carID). |
|                                                                                                                                         |                                                                                                                                                                                                     |

## Mange Payment

| Actors' Interactions                                                                             | System Responses                                                                                                 |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| 1. User submits<br>validateSession(token) to<br>Auth Mgmt.                                       | 2. Auth Mgmt validates<br>session(token).                                                                        |
|                                                                                                  | 3. Auth Mgmt returns<br>sessionStatus(isValid).                                                                  |
| 4. Session confirmed if valid.                                                                   | 5. System returns<br>sessionConfirmed(isValid) to User.                                                          |
| 6. User (session valid)<br>submits<br>createPayment(paymentInfo)<br>directly to Payment Service. | 7. Payment Service forwards<br>createPayment(paymentInfo) to Main<br>Controller for orchestration.               |
|                                                                                                  | 8. Main Controller processes and<br>returns orchestrationResult to<br>Payment Service.                           |
|                                                                                                  | 9. Payment Service processes<br>createPayment(paymentInfo) and<br>returns<br>createPaymentResult(paymentID).     |
|                                                                                                  | 10. Payment Service returns<br>paymentCreated(paymentID) to User.                                                |
| 11. User requests<br>verifyPayment(paymentID)<br>directly to Payment Service.                    | 12. Payment Service forwards<br>verifyPayment(paymentID) to Main<br>Controller for verification<br>coordination. |

|                                                                                           |                                                                                                                           |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
|                                                                                           | <b>13. Main Controller coordinates and returns coordResult to Payment Service.</b>                                        |
|                                                                                           | <b>14. Payment Service processes verifyPayment(paymentID) and returns verifyPaymentResult(verifiedStatus).</b>            |
|                                                                                           | <b>15. Payment Service returns paymentVerification(verifiedStatus) to User.</b>                                           |
| <b>16. User requests calculateFees(paymentData, feeType) directly to Payment Service.</b> | <b>17. Payment Service forwards calculateFees(paymentData, feeType) to Main Controller for fee computation oversight.</b> |
|                                                                                           | <b>18. Main Controller oversees and returns oversightResult to Payment Service.</b>                                       |
|                                                                                           | <b>19. Payment Service processes calculateFees(paymentData, feeType) and returns calculateFeesResult(feeBreakdown).</b>   |
|                                                                                           | <b>20. Payment Service returns feeBreakdown(feeBreakdown) to User.</b>                                                    |
| <b>21. User requests getPaymentStatus(paymentID) directly to Payment Service.</b>         | <b>22. Payment Service forwards getPaymentStatus(paymentID) to Main Controller for status aggregation.</b>                |
|                                                                                           | <b>23. Main Controller aggregates and returns aggregateResult to Payment Service.</b>                                     |

|                                                                                                    |                                                                                                                              |
|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                    | <b>24. Payment Service retrieves and returns<br/>getPaymentStatusResult(status).</b>                                         |
|                                                                                                    | <b>25. Payment Service returns<br/>paymentStatus(status) to User.</b>                                                        |
| <b>26. User requests<br/>generateReceipt(paymentID)<br/>directly to Payment Service.</b>           | <b>27. Payment Service forwards<br/>generateReceipt(paymentID) to Main<br/>Controller for receipt templating.</b>            |
|                                                                                                    | <b>28. Main Controller templates and<br/>returns templateResult to Payment<br/>Service.</b>                                  |
|                                                                                                    | <b>29. Payment Service processes<br/>generateReceipt(paymentID) and<br/>returns<br/>generateReceiptResult(receiptDoc).</b>   |
|                                                                                                    | <b>30. Payment Service returns<br/>receiptGenerated(receiptDoc) to<br/>User.</b>                                             |
| <b>31. User requests<br/>processRefund(paymentID,<br/>reason) directly to Payment<br/>Service.</b> | <b>32. Payment Service forwards<br/>processRefund(paymentID, reason) to<br/>Main Controller for refund approval.</b>         |
|                                                                                                    | <b>33. Main Controller approves and<br/>returns approvalResult to Payment<br/>Service.</b>                                   |
|                                                                                                    | <b>34. Payment Service processes<br/>processRefund(paymentID, reason)<br/>and returns<br/>processRefundResult(refundID).</b> |
|                                                                                                    | <b>35. Payment Service returns<br/>refundProcessed(refundID) to User.</b>                                                    |

**Mange Feedback**

| <b>Actors' Interactions</b> | <b>System Responses</b> |
|-----------------------------|-------------------------|
|-----------------------------|-------------------------|

|                                                                                                        |                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. User submits<br/>validateSession(token) to Auth<br/>Mgmt.</b>                                    | <b>2. Auth Mgmt validates session(token).</b>                                                                                                                 |
|                                                                                                        | <b>3. Auth Mgmt returns sessionStatus(isValid).</b>                                                                                                           |
| <b>4. Session confirmed if valid.</b>                                                                  | <b>5. System returns sessionConfirmed(isValid) to<br/>User.</b>                                                                                               |
| <b>6. User (session valid) submits<br/>createFeedback(feedbackData)<br/>directly to Feedback Mgmt.</b> | <b>7. Feedback Mgmt forwards<br/>createFeedback(feedbackData) to Main<br/>Controller.</b>                                                                     |
|                                                                                                        | <b>8. Main Controller processes<br/>createFeedback(feedbackData) and returns<br/>createFeedbackResult(feedbackID) to Feedback<br/>Mgmt.</b>                   |
|                                                                                                        | <b>9. Feedback Mgmt returns<br/>feedbackCreated(feedbackID) to User.</b>                                                                                      |
| <b>10. User submits<br/>acceptFeedback(feedbackID)<br/>directly to Feedback Mgmt.</b>                  | <b>11. Feedback Mgmt forwards<br/>acceptFeedback(feedbackID) to Main Controller.</b>                                                                          |
|                                                                                                        | <b>12. Main Controller processes<br/>acceptFeedback(feedbackID) and returns<br/>acceptFeedbackResult(moderationConfirmation)<br/>to Feedback Mgmt.</b>        |
|                                                                                                        | <b>13. Feedback Mgmt returns<br/>feedbackAccepted(moderationConfirmation) to<br/>User.</b>                                                                    |
| <b>14. User submits<br/>rejectFeedback(feedbackID,<br/>reason) directly to Feedback<br/>Mgmt.</b>      | <b>15. Feedback Mgmt forwards<br/>rejectFeedback(feedbackID, reason) to Main<br/>Controller.</b>                                                              |
|                                                                                                        | <b>16. Main Controller processes<br/>rejectFeedback(feedbackID, reason) and returns<br/>rejectFeedbackResult(rejectionConfirmation) to<br/>Feedback Mgmt.</b> |



|                                                                                        |                                                                                                                                            |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                        | <b>17. Feedback Mgmt returns feedbackRejected(rejectionConfirmation) to User.</b>                                                          |
| <b>18. User submits updateFeedback(feedbackID, newData) directly to Feedback Mgmt.</b> | <b>19. Feedback Mgmt forwards updateFeedback(feedbackID, newData) to Main Controller.</b>                                                  |
|                                                                                        | <b>20. Main Controller processes updateFeedback(feedbackID, newData) and returns updateFeedbackResult(updateSummary) to Feedback Mgmt.</b> |
|                                                                                        | <b>21. Feedback Mgmt returns feedbackUpdated(updateSummary) to User.</b>                                                                   |
| <b>22. User requests listFeedbacks(filters) directly to Feedback Mgmt.</b>             | <b>23. Feedback Mgmt forwards listFeedbacks(filters) to Main Controller.</b>                                                               |
|                                                                                        | <b>24. Main Controller processes listFeedbacks(filters) and returns listFeedbacksResult(filteredFeedbackList) to Feedback Mgmt.</b>        |
|                                                                                        | <b>25. Feedback Mgmt returns filteredFeedbackList to User.</b>                                                                             |

## Manage Car Buying

| <b>Actors' Interactions</b>                                           | <b>System Responses</b>                                                            |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>1. Buyer submits registration(profileData) to Main Controller.</b> | <b>2. Main Controller forwards createUserProfile(profileData) to Profile Mgmt.</b> |
|                                                                       | <b>3. Profile Mgmt creates user profile(profileData) and returns</b>               |

|                                                                                        |                                                                                                                                        |
|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                        | <b>createUserProfileResult(profileID) to Main Controller.</b>                                                                          |
|                                                                                        | <b>4. Main Controller triggers email verification: forwards verifyEmail(userID, verificationCode) to Auth Mgmt.</b>                    |
| <b>5. Buyer verifies email(userID, verificationCode) to Auth Mgmt.</b>                 | <b>6. Auth Mgmt verifies email(userID, verificationCode).</b>                                                                          |
|                                                                                        | <b>7. Auth Mgmt returns emailVerificationStatus(isVerified) to Main Controller.</b>                                                    |
|                                                                                        | <b>8. Main Controller confirms verification(isVerified) to Buyer.</b>                                                                  |
|                                                                                        | <b>9. System sends initial password(newPassword) via email to Buyer.</b>                                                               |
| <b>10. Buyer changes password(userID, newPassword) to Main Controller.</b>             | <b>11. Main Controller forwards changePassword(userID, newPassword) to Profile Mgmt.</b>                                               |
|                                                                                        | <b>12. Profile Mgmt changes password(userID, newPassword) and returns changePasswordStatus(changeConfirmation) to Main Controller.</b> |
|                                                                                        | <b>13. Main Controller returns changePasswordStatus(changeConfirmation) to Buyer.</b>                                                  |
| <b>14. (Optional) Buyer requests password change status update to Main Controller.</b> | <b>15. Main Controller updates password change status(changeConfirmation) via Profile Mgmt.</b>                                        |
| <b>16. Buyer requests searchCars(criteria)</b>                                         | <b>17. Buying Service forwards searchCars(criteria) to Main Controller.</b>                                                            |

|                                                                                     |                                                                                                              |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>directly to Buying Service.</b>                                                  |                                                                                                              |
|                                                                                     | <b>18. Main Controller performs externalSearch(criteria).</b>                                                |
|                                                                                     | <b>19. Main Controller validates external search(criteria).</b>                                              |
|                                                                                     | <b>20. Main Controller searches cars(criteria) and returns searchCarsResult(listings) to Buying Service.</b> |
|                                                                                     | <b>21. Buying Service displays listings(listings) to Buyer.</b>                                              |
| <b>22. Buyer requests getCarDetails(carID) directly to Buying Service.</b>          | <b>23. Buying Service forwards getCarDetails(carID) to Main Controller.</b>                                  |
|                                                                                     | <b>24. Main Controller syncs car details(carID).</b>                                                         |
|                                                                                     | <b>25. Main Controller returns getCarDetailsResult(carData) to Buying Service.</b>                           |
|                                                                                     | <b>26. Buying Service shows car details(carData) to Buyer.</b>                                               |
| <b>27. Buyer sends offer(buyerID, carID, offerData) directly to Buying Service.</b> | <b>28. Buying Service forwards sendOffer(buyerID, carID, offerData) to Main Controller.</b>                  |
|                                                                                     | <b>29. Main Controller validates offer(offerData).</b>                                                       |
|                                                                                     | <b>30. Main Controller submits offer to external(offerData).</b>                                             |
|                                                                                     | <b>31. External service returns sendOfferResult(offerID).</b>                                                |
|                                                                                     | <b>32. Main Controller returns sendOfferResult(offerID) to Buying Service.</b>                               |

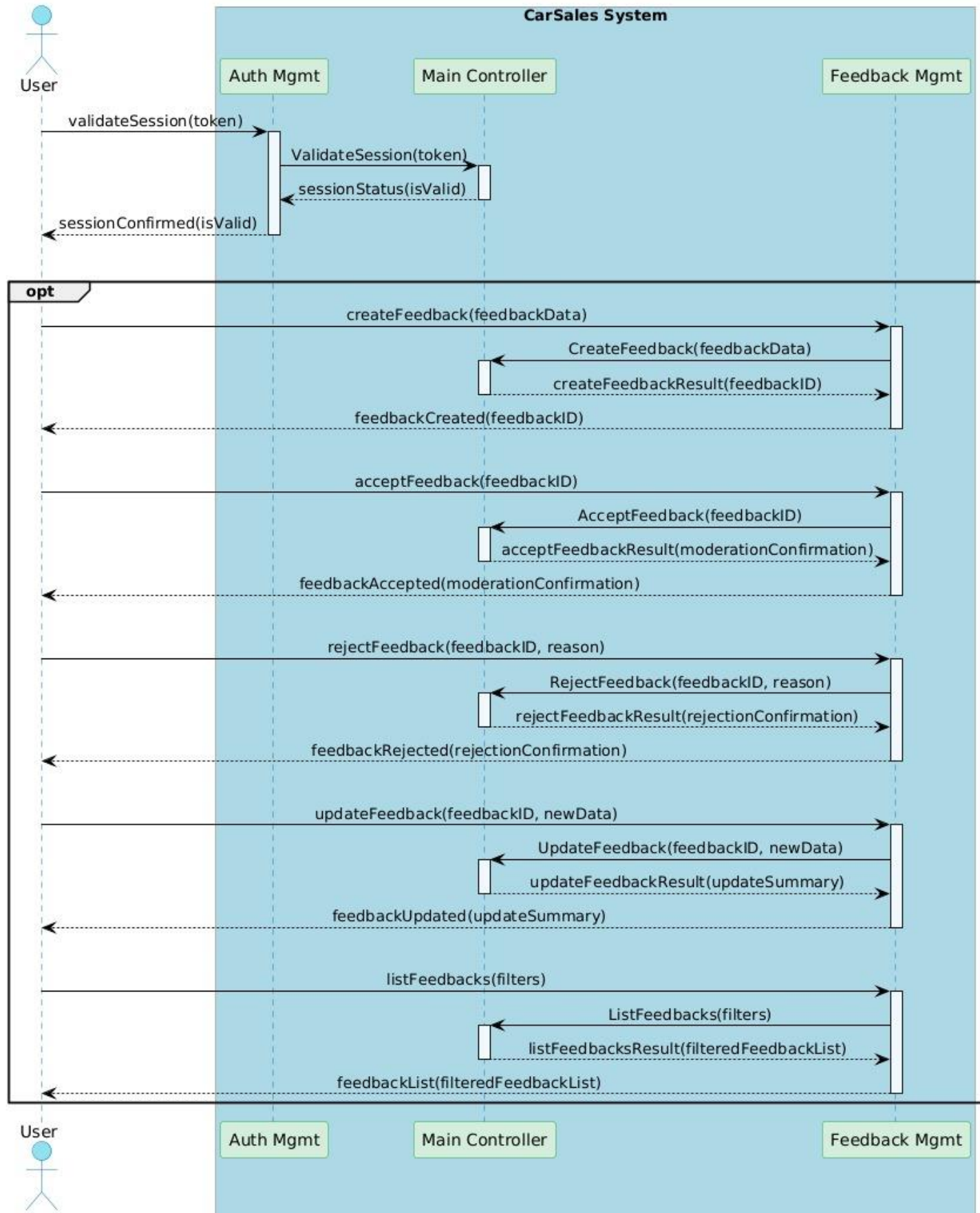
|                                                                                       |                                                                                                                    |
|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
|                                                                                       | <b>33. Buying Service returns offerSubmitted(offerID) to Buyer.</b>                                                |
| <b>34. Buyer processes payment(amount, method) directly to Payment Service.</b>       | <b>35. Payment Service forwards processPayment(amount, method) to Main Controller.</b>                             |
|                                                                                       | <b>36. Main Controller processes payment(amount, method) and returns paymentsSuccess(txID) to Payment Service.</b> |
|                                                                                       | <b>37. Payment Service finalizes order(orderID, carID) via Buying Service.</b>                                     |
|                                                                                       | <b>38. Buying Service notifies buyer success(orderID).</b>                                                         |
| <b>39. Buyer views purchase confirmation(orderID).</b>                                | <b>40. Main Controller shows purchase confirmation(orderID) to Buyer.</b>                                          |
| <b>41. Buyer submits buyer review(orderID, reviewData) directly to Feedback Mgmt.</b> | <b>42. Feedback Mgmt forwards validateAndPostReview(orderID, reviewData) to Main Controller.</b>                   |
|                                                                                       | <b>43. Main Controller validates and saves review data.</b>                                                        |
|                                                                                       | <b>44. Main Controller returns reviewPostedSuccess to Feedback Mgmt.</b>                                           |
|                                                                                       | <b>45. Feedback Mgmt returns reviewPostedSuccess to Buyer.</b>                                                     |
| <b>46. Buyer requests get search history to Main Controller.</b>                      | <b>47. Main Controller forwards getSearchHistory() to Buying Service.</b>                                          |
|                                                                                       | <b>48. Buying Service returns historyList(found) to Main Controller.</b>                                           |

|  |                                                                 |
|--|-----------------------------------------------------------------|
|  | <b>49. Main Controller returns historyList(found) to Buyer.</b> |
|--|-----------------------------------------------------------------|

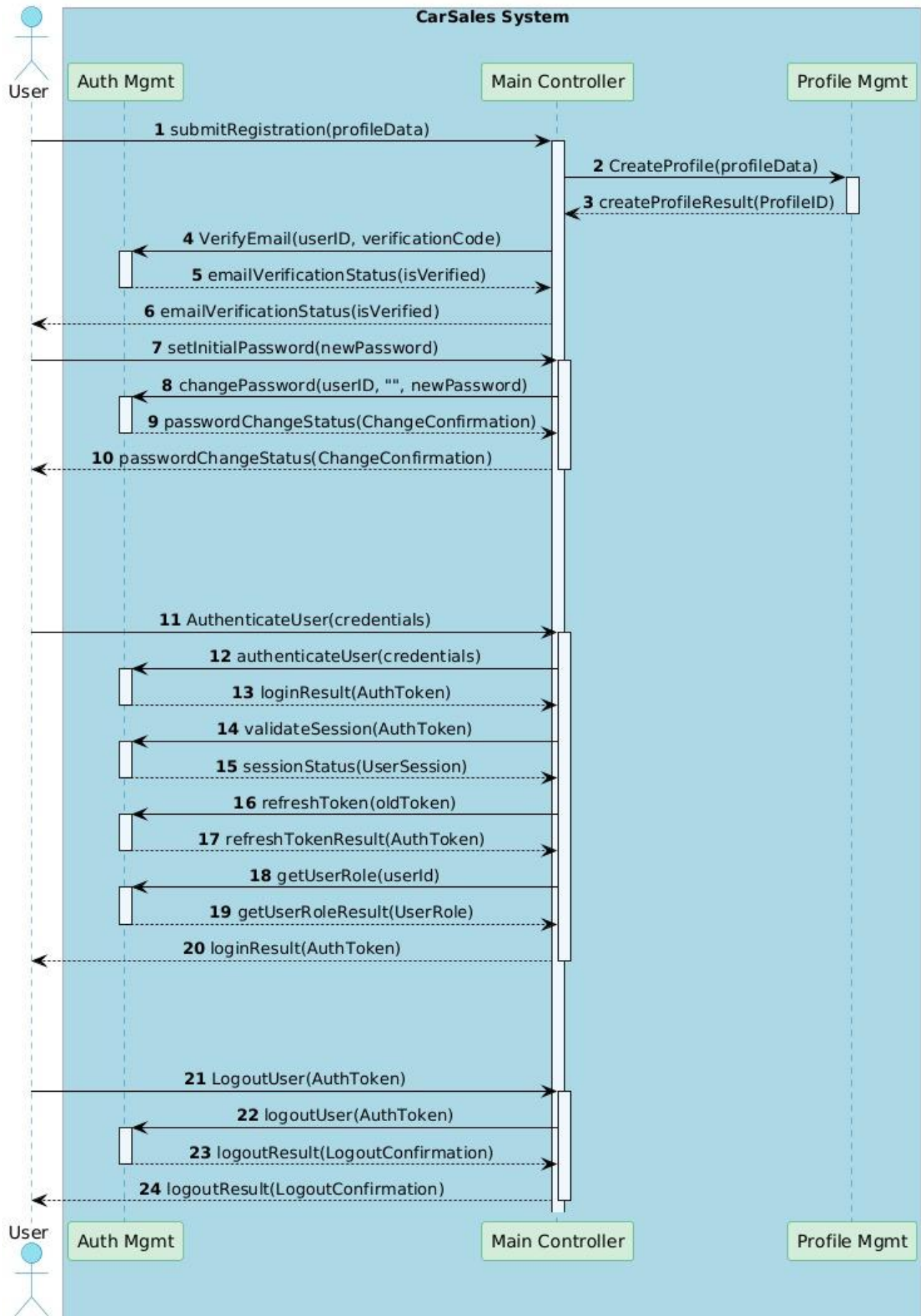
**3. Class diagrams:**  
**On the pdf**

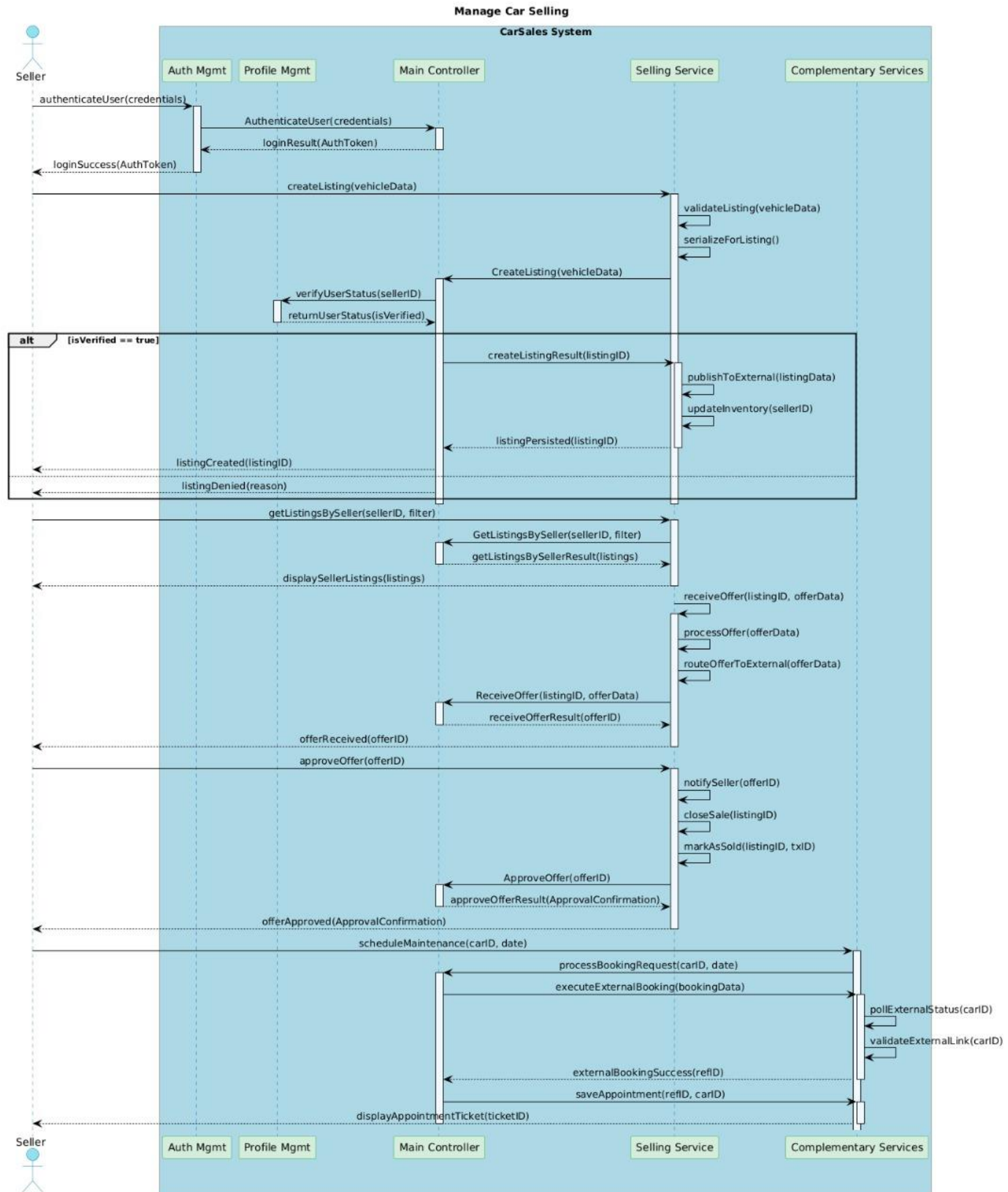
**4. Sequence diagrams:**

## Manage Feedback



## Manage Auth





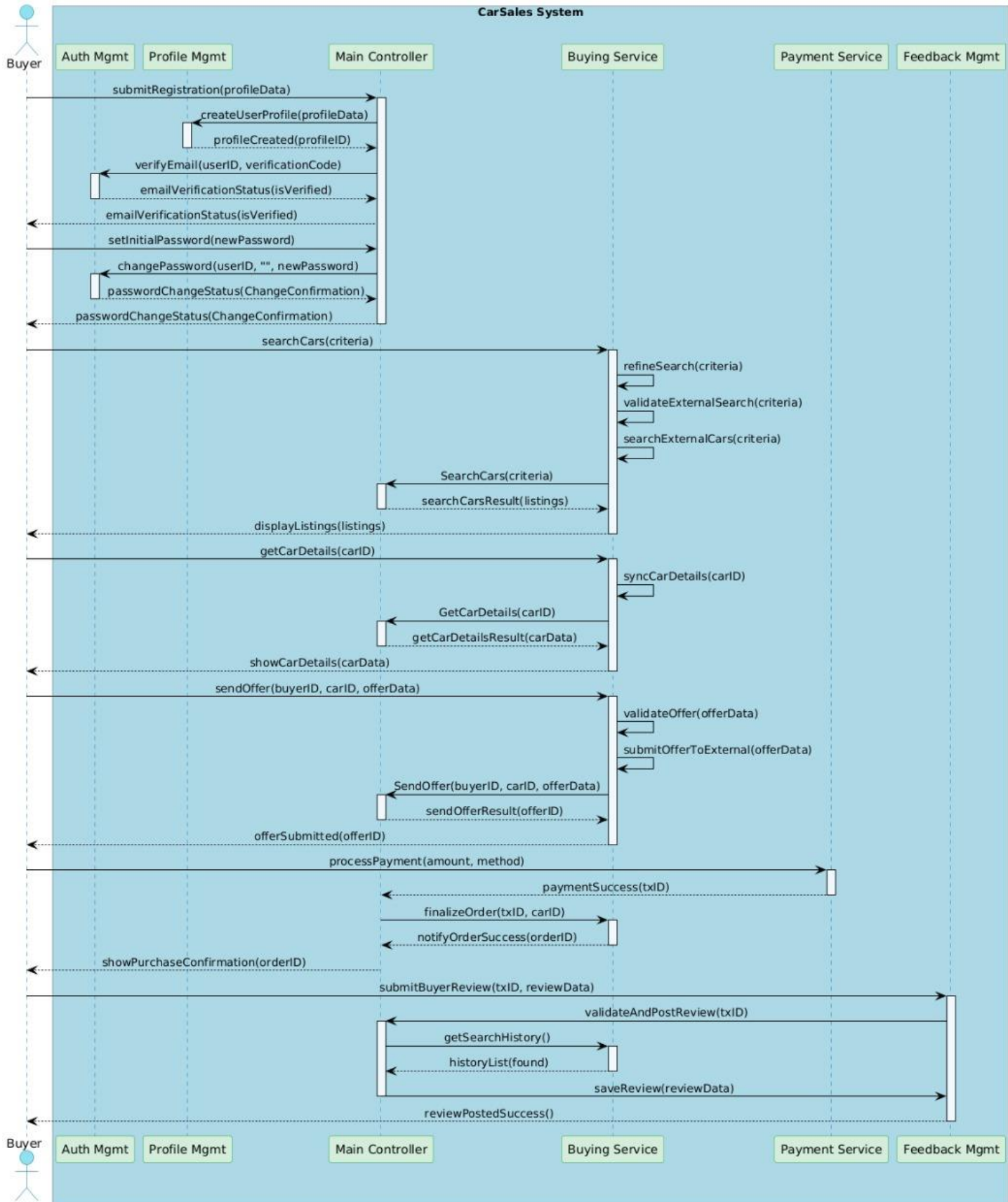


## Manage Payment



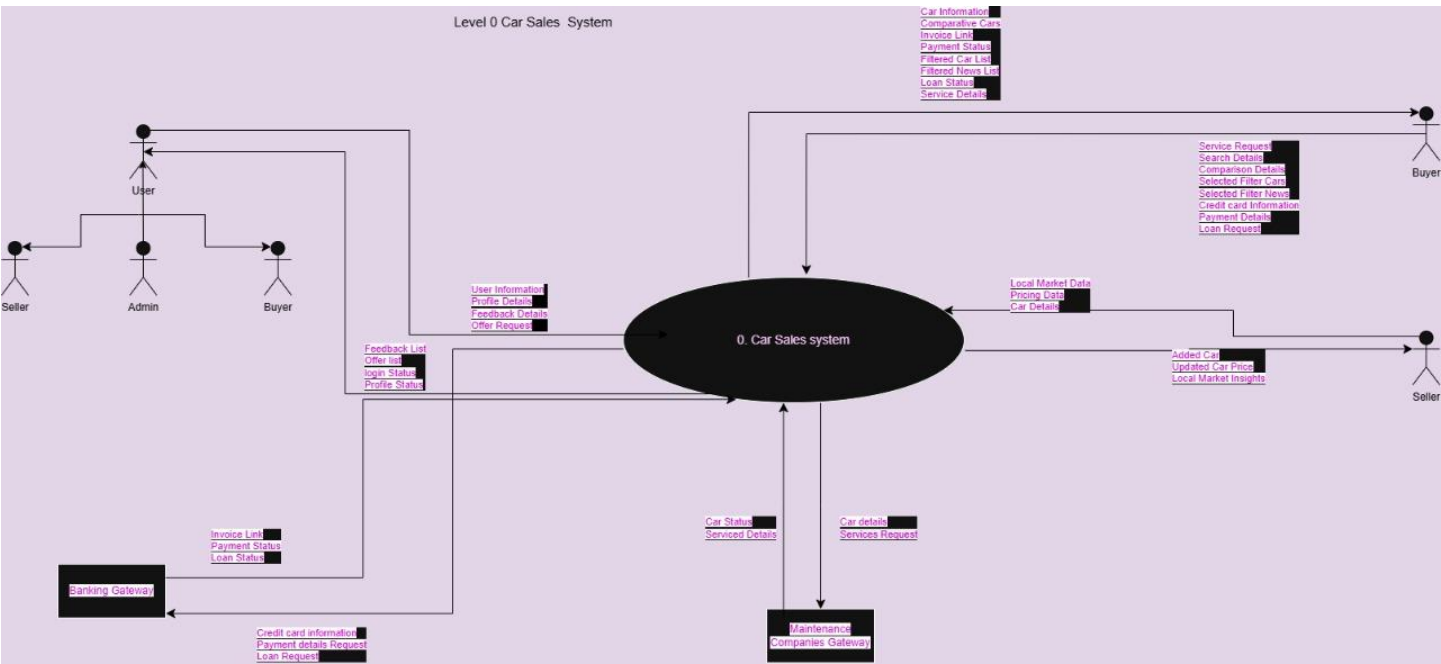
# Manage car buying

## CarSales System

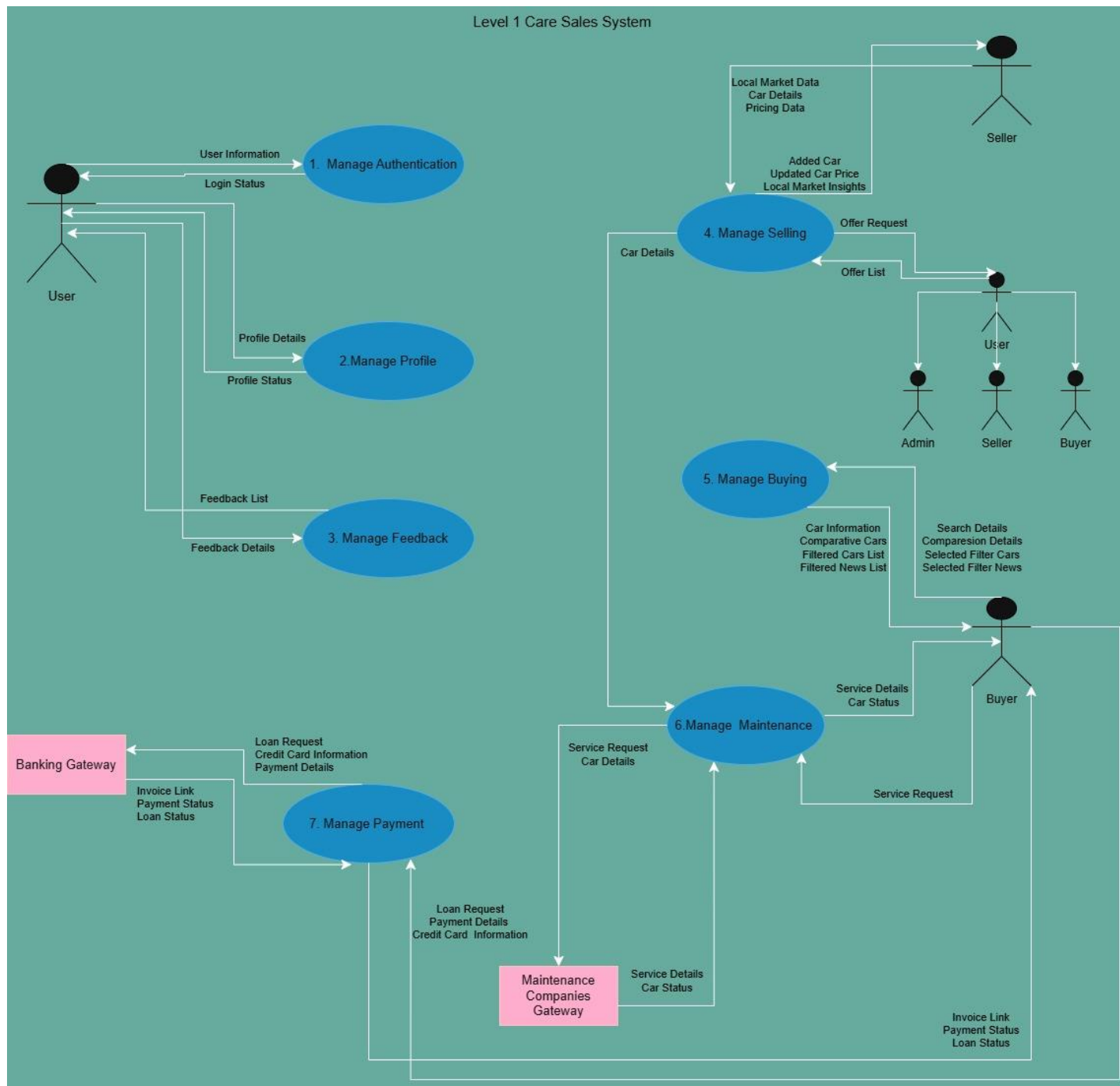


5. Dataflow diagrams (DFD):

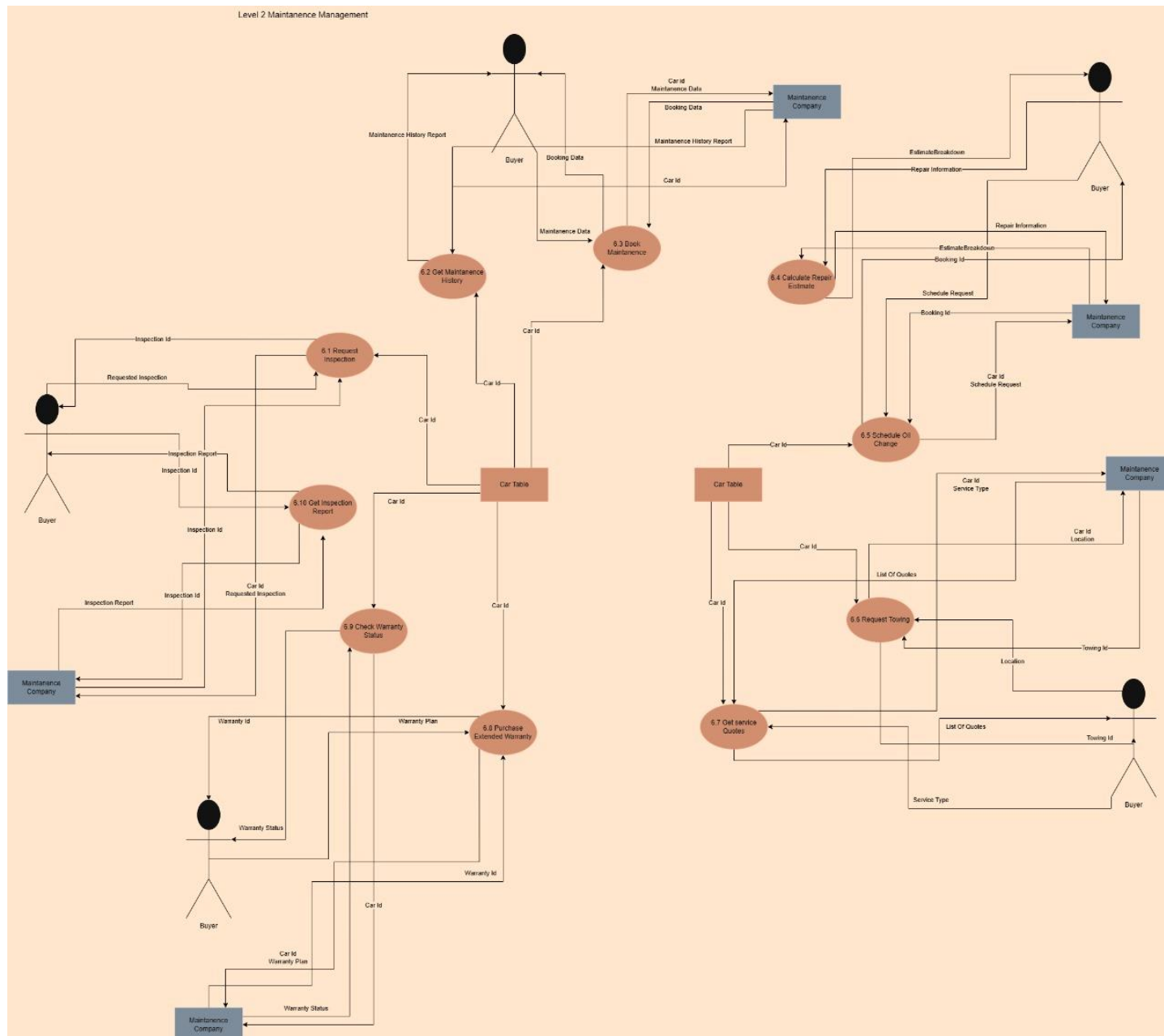
- Level0:



- Level1:

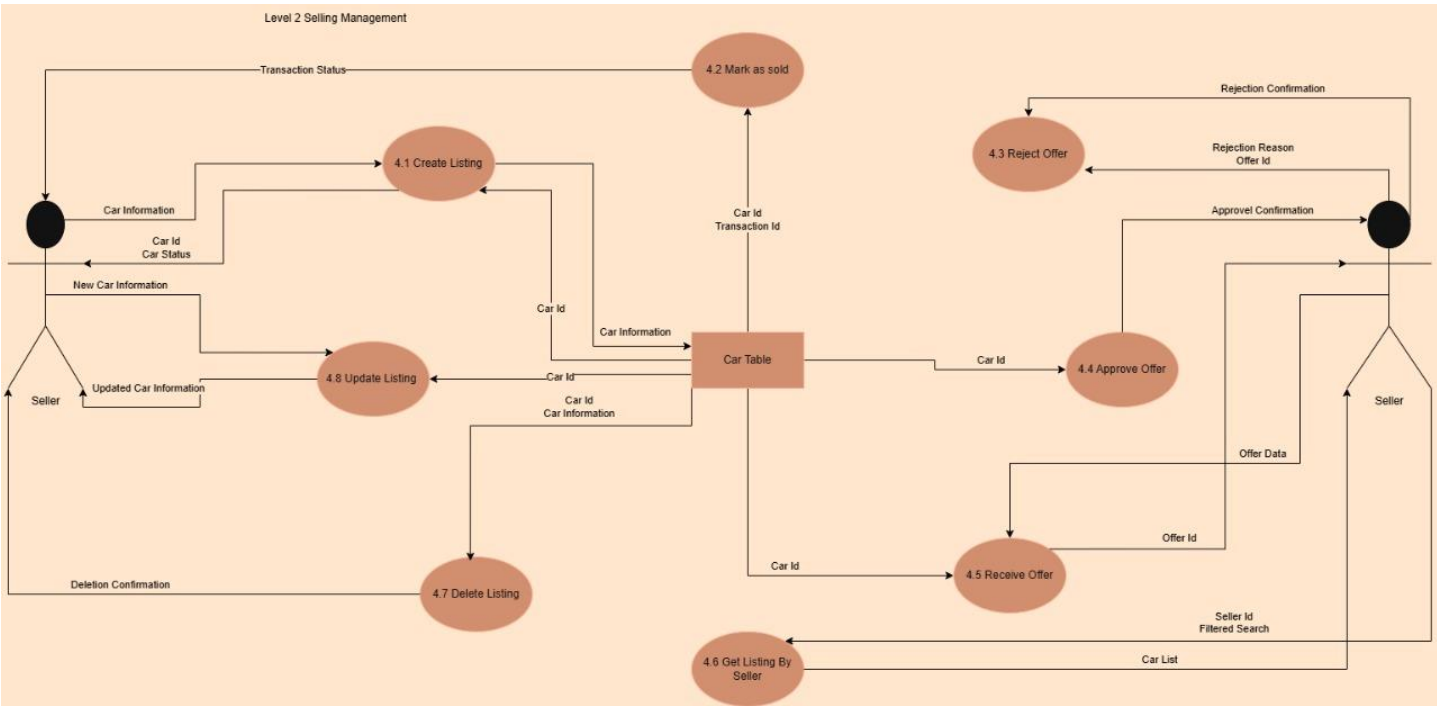
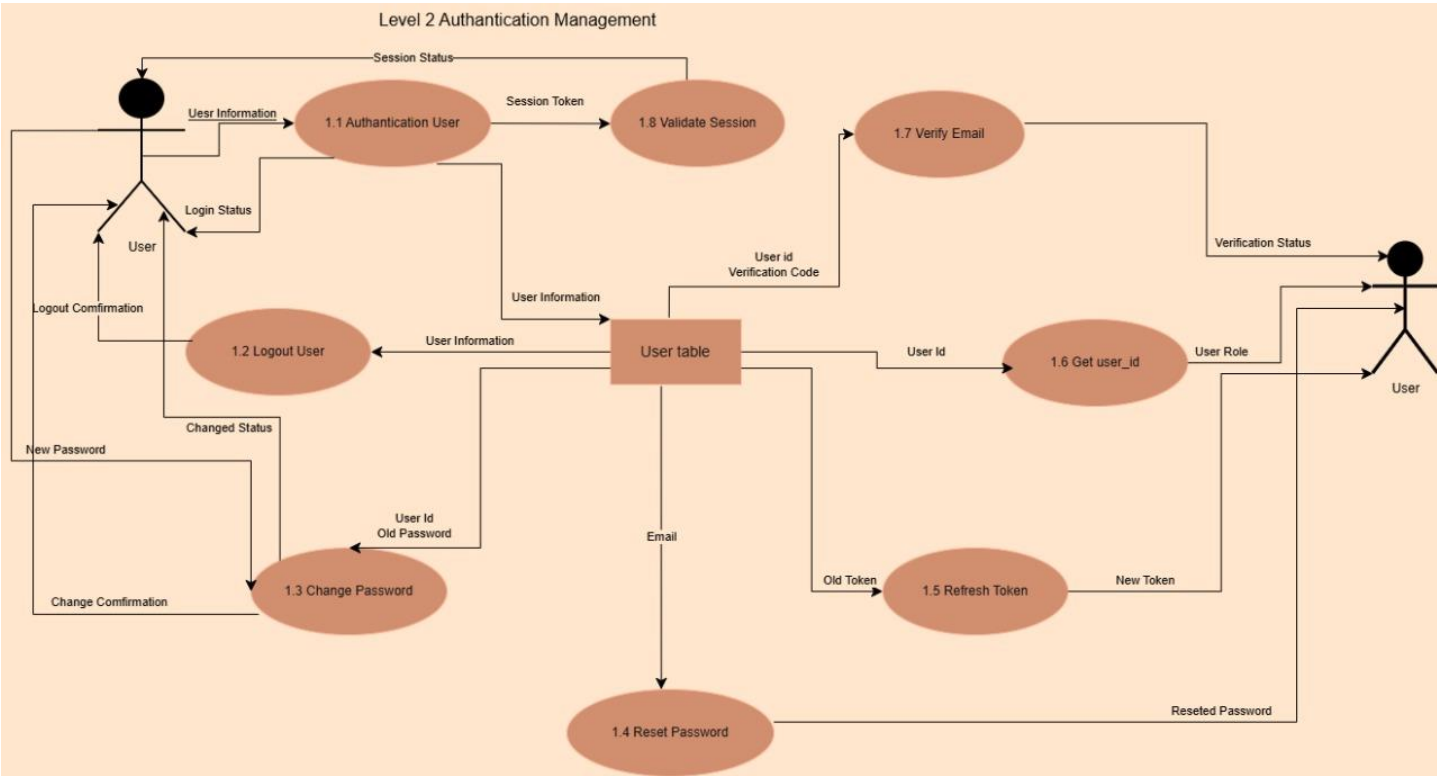


- Level2:

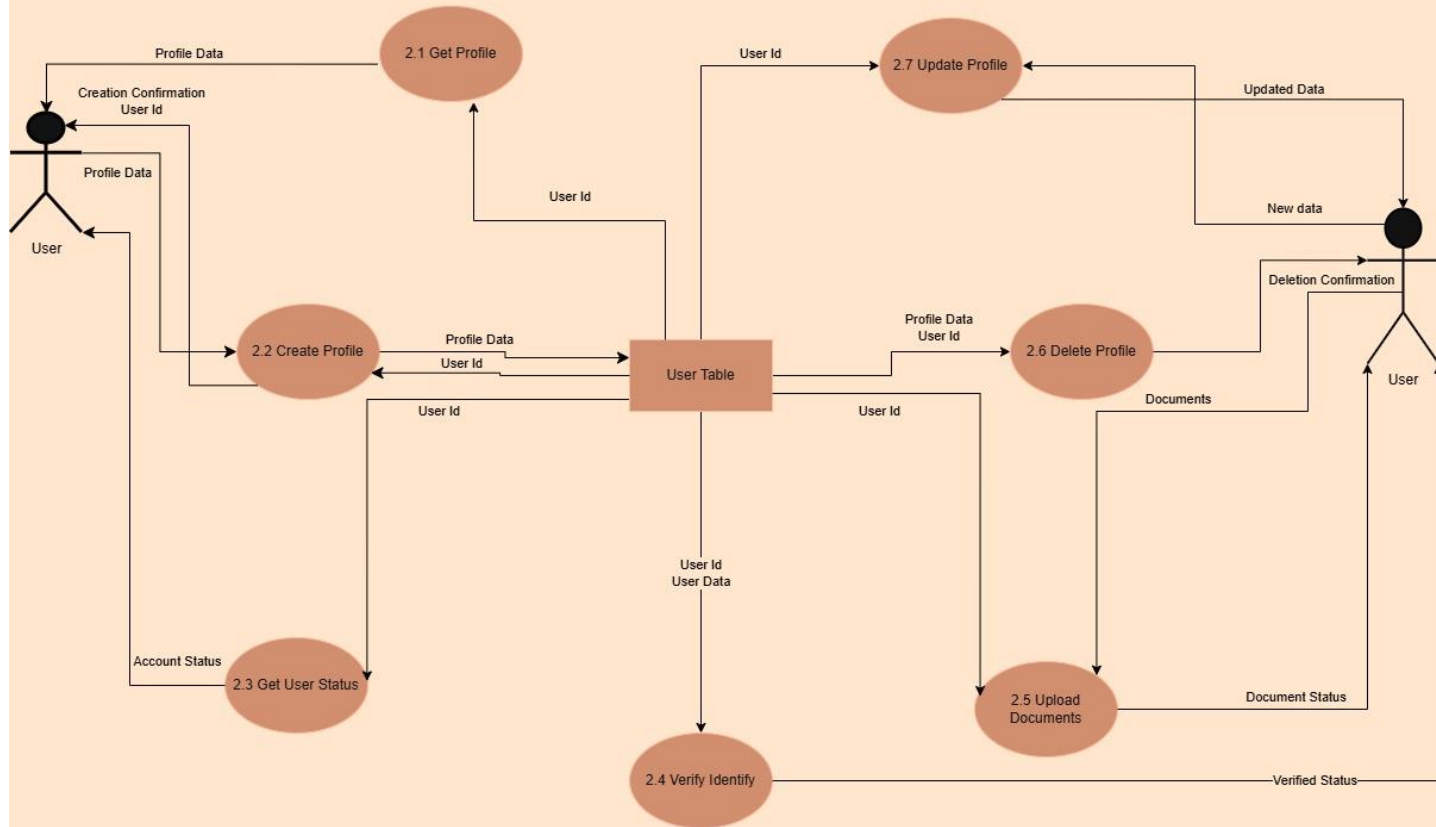








## Level 2 Profile Management



## Level 2 Payment Management

